Shimmer Activity Recognition

INTERNSHIP PROGRESS REPORT | DATA PREPARATION AND APP

Project objective

Prepare Shimmer sensor recordings for a future activity-classification model. The seven activity labels are Fall, Jump, Lay, Run, Sit, Stand, and Walk. The model-training stage has not yet been performed.

Completed workflow

Stage	What was completed
1. Load data	The app accepts MATLAB MAT files or CSV files in one upload. For individual activity files, the filename supplies the label, for example Walk.mat becomes Walk. The app displays sample/activity counts and a data preview.
2. Export data	Download a combined CSV, or download one CSV per activity in a ZIP when MATLAB files are uploaded.
3. Visualize signals	Plot the first 1,000 samples for the selected activity on X, Y, and Z, plus magnitude sqrt(X^2 + Y^2 + Z^2). Red dashed lines mark possible breaks in the displayed samples.
4. Inspect timestamps	Plot timestamp against sample number for a selected activity. Display the median positive time interval and the number of detected breaks when calculable.
5. Extract features	Create 128-sample non-overlapping windows within continuous segments, preview window features, filter the preview by activity, and download features_128.csv.

Data and processing details

Dataset structure

The MATLAB recordings contain a numeric data matrix. Its 11 columns are interpreted as timestamp, acceleration X, acceleration Y, acceleration Z, and seven additional sensor channels. The extra channels are retained under neutral names because their meanings have not been confirmed. The current plots and features use X/Y/Z only.

How the signal plots work

- The app sorts samples by sample index and displays up to 1,000 samples for the selected activity.
- X, Y, and Z are plotted separately so directional information stays visible.
- Magnitude is calculated sample by sample as sqrt(X^2 + Y^2 + Z^2). It summarizes the combined acceleration size, not its direction.
- The saved plots use sample index on the horizontal axis. The timestamp diagnostic uses sample number horizontally and timestamp as stored vertically.

Segment breaks and windowing

When segment IDs are absent, the app infers a new segment if a timestamp difference is non-positive or greater than 1.5 times the median positive interval. Existing segment IDs are kept. Each feature window stays within one segment, so it does not cross a detected break. This is a heuristic; irregular data could be flagged, and the threshold should be confirmed against the sensor documentation or supervisor guidance.

Features per window

Axis	Features calculated
X	Maximum, minimum, mean, standard deviation
Y	Maximum, minimum, mean, standard deviation
Z	Maximum, minimum, mean, standard deviation

This gives 12 numeric features per complete window. The activity is the target label. Segment ID and starting sample are traceability metadata, not model inputs. Incomplete windows and windows with non-finite X/Y/Z values are skipped.

Window duration

128 samples equal 2.56 seconds at 50 Hz. The observed median timestamp interval was about 19.53 ms; if the timestamps are in milliseconds, the implied rate is about 51.2 Hz and the window lasts about 2.50 seconds. The app follows the requested 128-sample size; timestamp units and nominal rate remain unconfirmed.

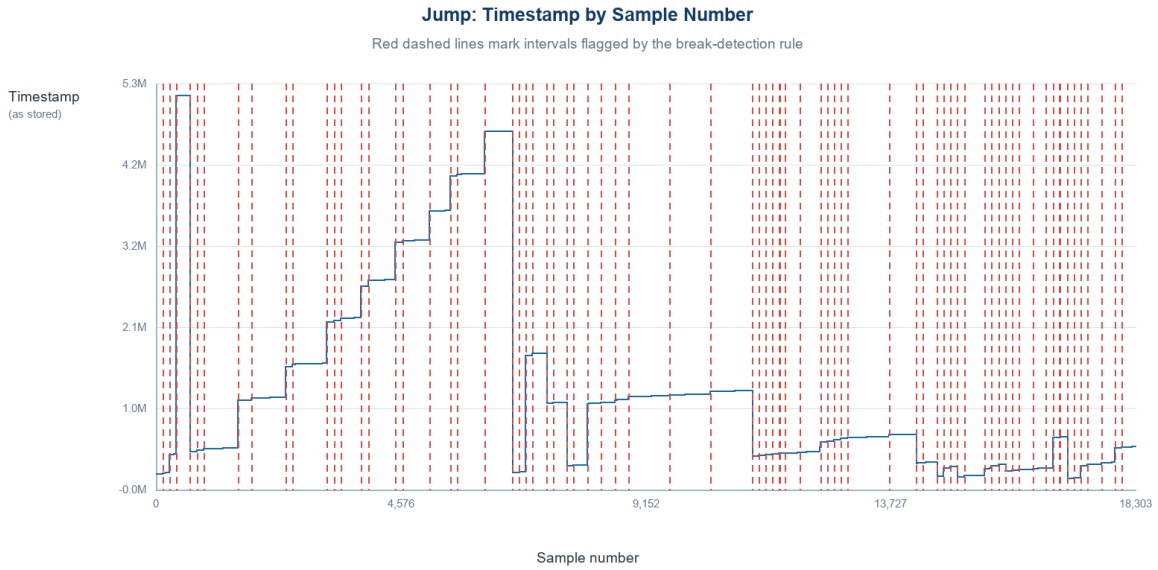
Current project status

Completed: data loading/export, visualization, timestamp inspection, segmentation-aware windowing, statistical feature extraction, and app interface. Not yet done: training or evaluating machine-learning models.

Timestamp and feature extraction examples

Timestamp diagnostic for Jump

The chart plots the stored timestamp against the sample number for the Jump activity. The red dashed markers show positions flagged by the current heuristic; they are possible breaks, not verified sensor failures.



Median positive interval	Intervals flagged by rule
19.53 timestamp units	81

Example of extracted window features

The table below shows the 12 values from one complete Walk window: segment 0, starting at sample 0, containing 128 samples. These are example feature values, not model predictions. Sensor units have not been confirmed.

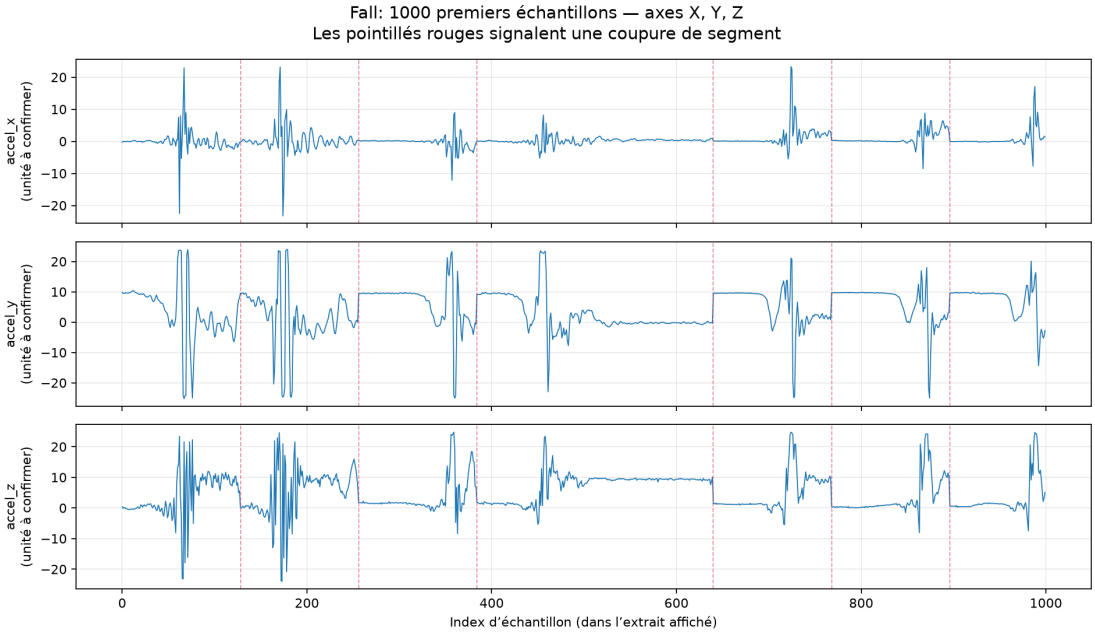
Axis	Maximum	Minimum	Mean	Standard deviation
X	5.5351	-3.3039	0.9014	1.6521
Y	15.1394	5.3903	9.7873	1.9215
Z	2.4624	-4.5658	-0.4439	1.4868

The four statistics are computed independently for each axis over the 128 raw samples. One row of the downloaded feature table corresponds to one window.

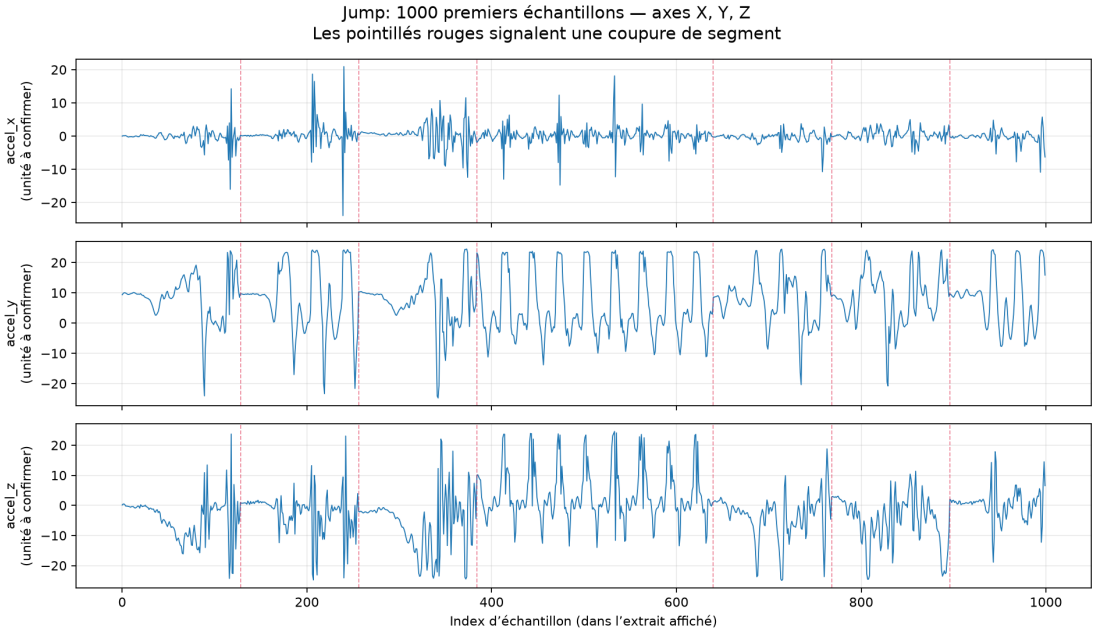
Signal visualization | Activities 1-2

First 1,000 samples for each activity. The three traces show X, Y, and Z. Dashed red lines indicate segment boundaries in the excerpt; axis units are not confirmed.

Fall



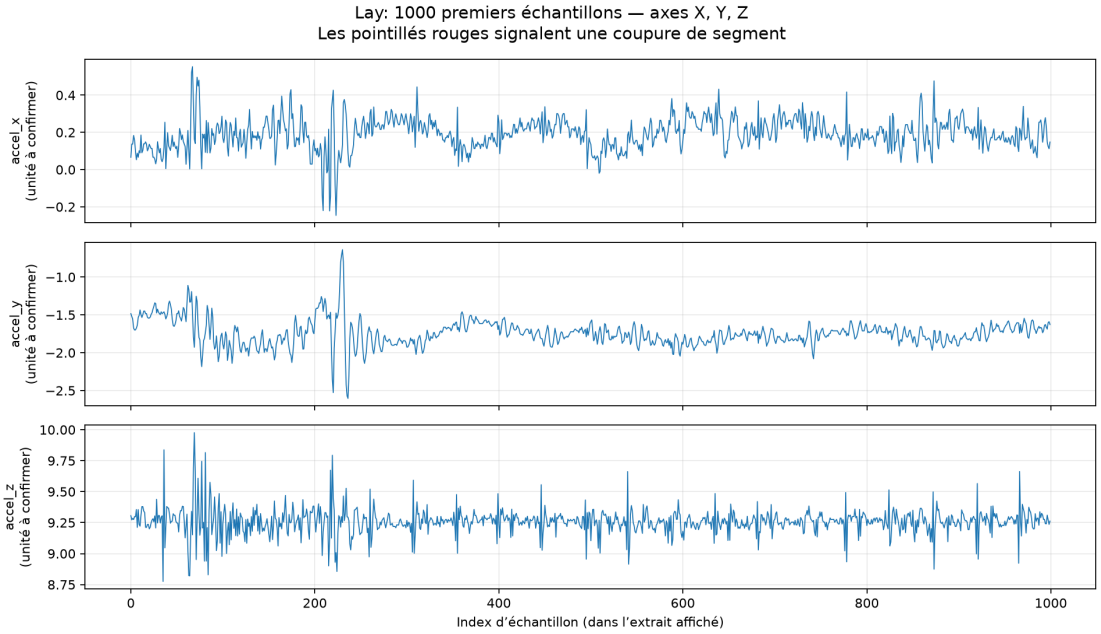
Jump



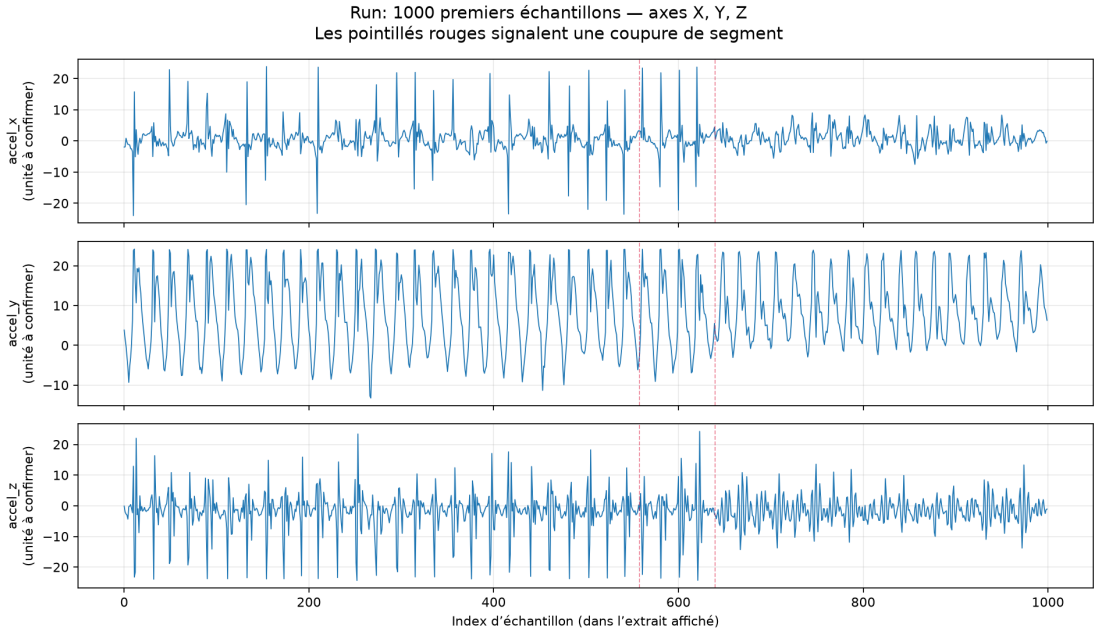
Signal visualization | Activities 3-4

First 1,000 samples for each activity. The three traces show X, Y, and Z. Dashed red lines indicate segment boundaries in the excerpt; axis units are not confirmed.

Lay



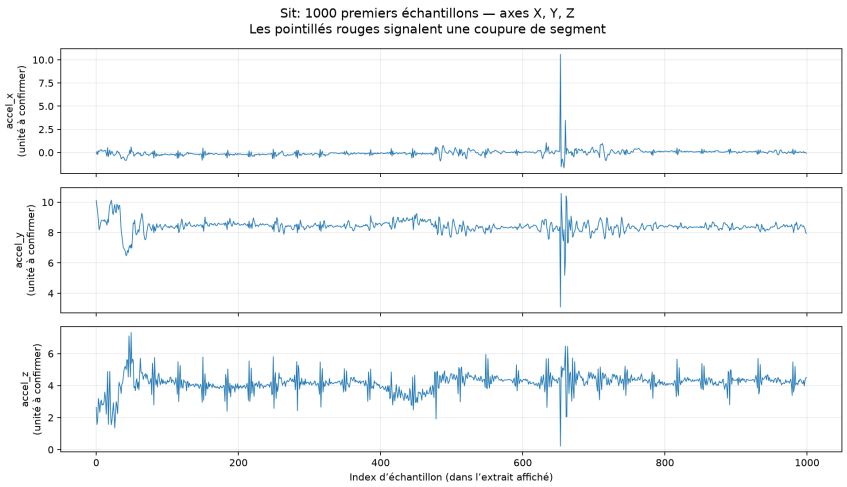
Run



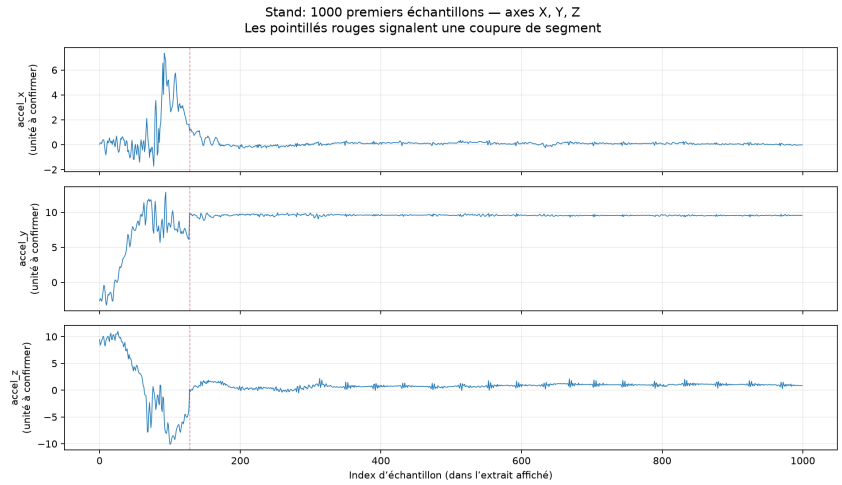
Signal visualization | Activities 5-7

First 1,000 samples for each activity. The three traces show X, Y, and Z. Dashed red lines indicate segment boundaries in the excerpt; axis units are not confirmed.

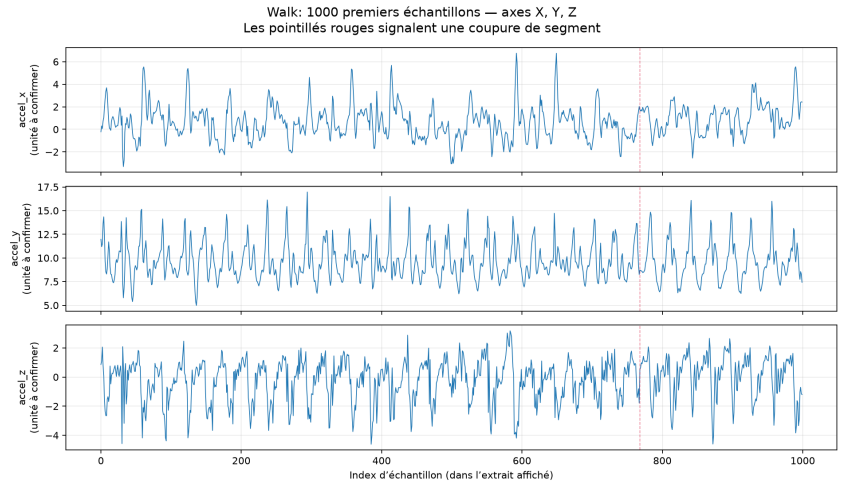
Sit



Stand

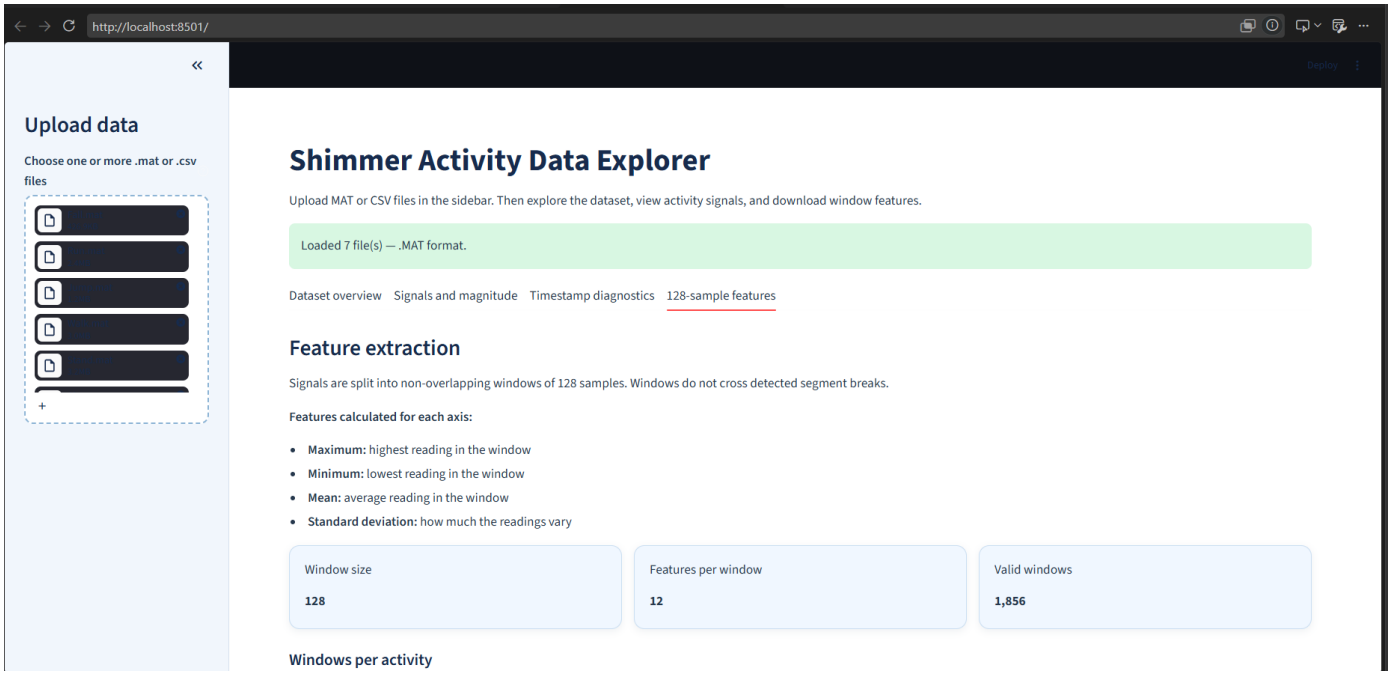


Walk



App Feature Extraction Screen

This screenshot documents the feature-extraction view of the interactive app with seven uploaded MATLAB activity files.



What the screen shows: non-overlapping windows of 128 samples; four statistics (maximum, minimum, mean, and standard deviation) for each of X, Y, and Z; 12 features per window; and 1,856 valid windows in the displayed dataset.

The 1,856 count is the value shown by the app for the files loaded in this screenshot. Each complete window contributes one feature row.

Timestamp continuity and window validation

How the current implementation works

The app sorts each activity recording by sample index and respects any existing segment boundaries. It also checks the timestamps. A new provisional boundary is flagged when time does not advance, a timestamp is missing, or an interval exceeds 1.5 times the median positive interval within the recording segment.

Feature extraction then creates non-overlapping windows of 128 samples inside the resulting segments. Each candidate window is checked to ensure its timestamps are present, strictly increasing, and do not contain an interval above the current threshold. Windows with non-finite X/Y/Z values are also excluded.

These checks reduce the chance of combining samples across a timing discontinuity. The threshold is a heuristic: flagged points are possible breaks, not confirmed sensor interruptions. Timestamp units and the appropriate threshold still need confirmation from dataset documentation or the supervisor.

Latest feature-extraction run

Result	Value
Window size	128 samples
Features per window	12 (max, min, mean, standard deviation for X, Y, Z)
Valid windows generated	1,856
Windows rejected by timestamp continuity check	0
Windows rejected for non-finite X/Y/Z values	0

These counts are from the user-reported run of python extract_features.py on the current all_activities.csv file. A zero count means no complete candidate window failed the implemented checks in that run; it does not independently prove that all sensor timing is physically continuous.